

An EKF Radar-Camera Tracking Framework with Fog-Aware Adaptive Camera Covariance Scaling

Shayma Alteneiji
Electrical Engineering
Khalifa University
Abu Dhabi, United Arab
Emirates
100063072@ku.ac.ae

Engy Farouk
Electrical Engineering
Khalifa University
Abu Dhabi, United Arab
Emirates
100061659@ku.ac.ae

Maryam Alhmoudi
Electrical Engineering
Khalifa University
Abu Dhabi, United Arab
Emirates
100062844@ku.ac.ae

Hind Albastaki
Electrical Engineering
Khalifa University
Abu Dhabi, United Arab
Emirates
100060327@ku.ac.ae

Aleksej Makarov
Vlatacom Institute
Belgrade, Serbia
Aleksej@vlatacom.com

Mohamed Alkhatib
Research & Development
Mohammed Bin Rashid Space
Centre
Dubai, United Arab Emirates
mohamed.alkhatib@mbrsc.ae

Omar Alzaabi
Electrical Engineering
Khalifa University
Abu Dhabi, United Arab
Emirates
omar.alzaabi@ku.ac.ae

Ahmed Altunaiji
Electrical Engineering
Khalifa University
Abu Dhabi, United Arab
Emirates
ahmed.altunaiji@ku.ac.ae

Abstract—This paper presents the design and implementation of a radar–camera data fusion framework for multi-target tracking using an Extended Kalman Filter (EKF). The limitation of degraded camera accuracy under foggy weather is addressed by integrating a lightweight machine-learning (ML) model, a Random Forest, for fog-level estimation. The camera noise covariance is scaled in accordance with the ML model result, aiming to reduce the reliance on the received camera measurement during the EKF state estimation update step. Due to the multi-rate nature of the sensors, an asynchronous fusion architecture supporting radar-only, camera-only, or combined-measurement EKF updates, as well as explicit handling of out-of-sequence measurements, is implemented. Measurement-to-track association is performed using the Global Nearest Neighbour (GNN) algorithm. ArUco markers are used to extract target ground-truth states for the quantitative evaluation of the tracking accuracy. The radar-only EKF tracker achieved root mean square error (RMSE) values as low as 0.5 m in the tested scenario. The fog classifier achieved an accuracy of 97% on the testing dataset. Full validation of the fusion framework was limited by calibration-related projection errors attributed to the positioning of the setup. The paper demonstrates system design and integration, studied engineering trade-offs, calibration methodology, and highlights practical challenges in fusion systems.

Keywords—EKF, asynchronous sensor fusion, GNN, calibration, fog-level estimation, machine-learning model

I. INTRODUCTION

This paper presents a camera–radar data fusion framework with applications in Advanced Driver Assistance Systems (ADAS) [1], [2]. Radar and camera sensors provide complementary data from a given scene. With sensor fusion, each sensor can compensate for the weaknesses of the other. For example, the camera provides rich visual information and reliably detects objects under normal lighting but degrades under adverse weather conditions [1], [2]. Radar, on the other hand, provides range and velocity measurements, maintains

more consistent performance under adverse weather and low-light conditions, and can detect objects at longer ranges [1], [2].

The designed fusion framework targets processing platforms such as the Raspberry Pi due to their affordability and accessibility. Although many recent fusion and object-detection approaches rely on deep neural networks, these methods were deemed impractical for the selected platform; instead the Extended Kalman Filter (EKF) was selected, due to its ubiquity and relatively low computational intensity. The algorithm performs object-level fusion, i.e., processing the independent target detections of the radar and camera sensors before integration, in contrast to data-level fusion, for example, which acts on raw sensor data.

The paper addresses the challenge of degraded system performance under foggy conditions using a machine-learning (ML) model for adaptive camera covariance scaling. This was inspired by [3], which implemented a support vector machine (SVM) framework for fog-level estimation. The model was trained by associating a numerical fog-density label with six image features and used YOLOv4 to extract the additional training feature of the distribution of confidence scores of objects detected per frame.

In this paper, we propose integrating a lightweight Random Forest classifier to estimate the fog-level class based on seven image features, including the average confidence score of YOLO-detected objects. The confidence in the received camera data is calibrated by scaling the camera covariance matrix based on fog-level classification, aiming to improve target tracking accuracy by reducing the reliance of the tracker state estimator on degraded camera data. The main contributions of this paper are as follows:

- An asynchronous radar–camera tracking framework using a TI IWR6843 radar, HQ-IR Cut camera, and Raspberry Pi.

- A tracking pipeline supporting radar-only, camera-only, and combined updates with OOSM handling.
- Fog-aware camera covariance adaptation method using a Random Forest ML classification model.
- Methodology and experimental validation of radar tracking, calibration, and fog classification, as well as failure analysis of homography transformation implementation

II. SYSTEM ARCHITECTURE

Fig. 1 illustrates the camera–radar data fusion system block diagram. The pipeline is triggered upon the arrival of a new measurement from a sensor. The radar module employs a DSP unit, outputting high-level processed data. This data is then parsed, with the output including 3D object coordinates and speed measurements. On the Raspberry Pi, through multithreading, object detection on the camera module (YOLOv11) output is performed simultaneously, generating bounding boxes, centroids, and confidence scores for each detected object.

With each sensor using a different coordinate frame, alignment of the sensor perspectives is required. This is performed through coordinate transformation applied to the camera data using the intrinsic and extrinsic parameters obtained from offline calibration, unifying the point of view (PoV) to that of the radar.

Temporal alignment is then performed to order sensor measurements by their arrival times. If both sensors capture data corresponding to the same frame within a small delay window, the EKF update is performed using the combined measurement from both sensors. Otherwise, the EKF update is executed using a single sensor measurement (camera-only or radar-only).

Once the update flag is determined, the EKF state and measurement prediction step is executed, followed by data association using the Global Nearest Neighbour (GNN) method for measurement-to-track association. Subsequently, the track management logic layer, including initialization, deletion, and partial inactivation, is performed.

The next stage employs an integrated Random Forest model, which extracts features from the camera frames to classify the fog-level. Based on the fog-level class, the camera measurement covariance matrix is scaled accordingly. This process repeats for each new incoming measurement. Finally, tracks that accumulate more than some threshold value of assigned measurements are confirmed and displayed.

III. METHODOLOGY

A. Radar Module Configuration

The radar parameters, including the data transmission or update rate via the UART port [4], can be controlled via the configuration file which is sent to the radar module during deployment. The configuration file can be customized using Texas Instruments' mmWave_Demo_Visualizer browser-based application. For the desirable configuration, we opted for the best_range scene, as it allows the highest possible radial velocity of about 0.95–1 m/s [5], preventing the detection a single target

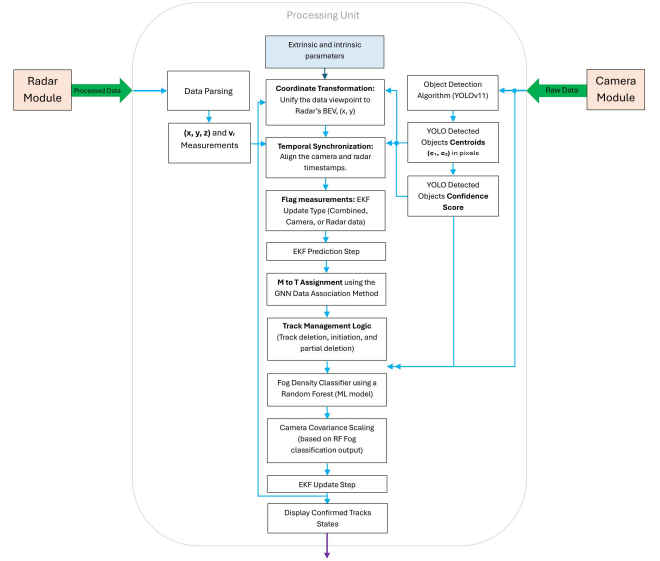


Fig. 1. High-level overview of the data fusion system.

that actually corresponds to two closely spaced pedestrians moving at similar speeds.

The summary of the chosen configuration file parameters is shown in Table I. Note that the radar update rate or FPS rate was kept at 10, the higher end of the recommended range, to avoid data loss during the limited interframe period of transmitting the captured data [4].

B. Calibration and Coordinate Transformation

With each sensor having a different PoV and using a different coordinate frame—specifically, the camera treats its optical centre as the origin, with its axes defined as z -forward, y -down, and x -right, while the radar uses the antenna as the origin, with its axes defined as z -up, y -forward, and x -right—the same targets viewed by each sensor would appear at different positions.

This is where calibration comes in. It provides the parameters for performing coordinate transformation, involving the rotation and translation of the coordinate frame of one sensor such that it matches the other, unifying the PoV.

The intrinsic camera calibration allows mapping 3D points in the camera frame to their respective 2D pixel coordinates [6]. The extrinsic calibration between the radar and camera enables the projection of the radar 3D coordinate points into the camera frame, or vice versa [6]. Combined, they are described by the general expression of perspective projection of 3D coordinates onto the image plane, as given by (1) [6], [7], [8]. The point (X, Y, Z) represents a real-world coordinate point. The $\mathbf{K}$ matrix includes the intrinsic parameters, while $[\mathbf{R} | \mathbf{t}]$ describes the rigid body transformation, or the extrinsic parameter matrix.

$$k \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{P}_{3 \times 4} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \mathbf{K}_{3 \times 4} [\mathbf{R} | \mathbf{t}]_{4 \times 4} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (1)$$

TABLE I. RADAR CONFIGURATION PARAMETRS

Parameter	Value
Frequency (GHz)	60
Bandwidth (GHz)	60 – 64
Scene Classifier	best_range
Azimuth Resolution(deg)	30+30
Maximum unambiguous Range(m)	15
Maximum Radial Velocity(m/s)	19.43
Radial velocity resolution(m/s)	0.31
Frame Duration(msec)	100

In this paper, the homography, or plane-to-plane transformation, is used instead of 2D-to-3D transformation, where the world coordinate points are assumed to all lie on the same plane ($Z = z_{projection}$) [6], [7], [8], as expressed by (2).

$$k \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{H}_{3 \times 3} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} \quad (2)$$

This is due to the limitations of the experimental setup, which uses a monocular camera that does not provide depth information. Moreover, it can be shown that the relation between the homography and projective transformation matrices is as given by (3). The details of obtaining the parameters are discussed in the methodology section.

$$\mathbf{H}_{3 \times 3} = [\mathbf{P}_{:,1} \quad \mathbf{P}_{:,2} \quad \mathbf{P}_{:,3} \times Z_{projection} + \mathbf{P}_{:,4}] \quad (3)$$

C. Asynchronous EKF Fusion

1) Motion Model

One of the simplest target manoeuvre models based on the known kinematic equations is the constant acceleration (CA) model, also referred to as the Wiener-process acceleration (WPA) or nearly constant acceleration (NCA) model [9]. A specific variant, the Wiener-sequence acceleration (WSA) model, has been used in a radar-camera data fusion implementations with EKF [2]. Compared to simpler motion models, such as constant velocity (CV), the CA WSA model strikes a balance between accuracy and complexity.

For two-dimensional motion, the original model lists the states as: $\mathbf{s} = (x, y, v_x, v_y, a_x, a_y)$ [9]. However, in our application, the measurements are uncoupled to the acceleration states a_x and a_y . These states are unobservable even after multiple dynamic steps and are therefore omitted from the model. The resulting $\mathbf{F}$ matrix is given by (4) [2], [9]:

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4)$$

The process noise covariance matrix, $\mathbf{Q}$, represents the deviation of the target from the modelled motion profile, the white acceleration noise assumption. The expression for the process noise covariance matrix is given by (5), and (6) [2], [9].

$$\mathbf{Q} = \text{var}(\mathbf{w}) \cdot E(\mathbf{w} \cdot \mathbf{w}^T) \quad (5)$$

$$\mathbf{Q} = \begin{bmatrix} \frac{\sigma_{ax}^2 \Delta t^4}{4} & 0 & \frac{\sigma_{ax}^2 \Delta t^3}{2} & 0 \\ 0 & \frac{\sigma_{ay}^2 \Delta t^4}{4} & 0 & \frac{\sigma_{ay}^2 \Delta t^3}{2} \\ \frac{\sigma_{ax}^2 \Delta t^3}{2} & 0 & \sigma_{ax}^2 \Delta t^2 & 0 \\ 0 & \frac{\sigma_{ay}^2 \Delta t^3}{2} & 0 & \sigma_{ay}^2 \Delta t^2 \end{bmatrix} \quad (6)$$

2) State Vector & Measurement Vectors and Equation

In our application, the tracked state vector is defined as given by (7). Moreover, the radar provides x , y , and v_r measurements, while the YOLOv11-based camera outputs u , v pixels of the detected bounding boxes centroids. Only the radial velocity is not linearly related to the states x , y , v_x and v_y , requiring the usage of the EKF. The equation given by (8) expresses the radial velocity as a function of the states [2].

$$\mathbf{s} = \begin{bmatrix} x \\ y \\ v_x \\ v_y \end{bmatrix} \quad (7)$$

$$v_r = \frac{xv_x + yv_y}{\sqrt{x^2 + y^2}} \quad (8)$$

To linearize this measurement, we apply the Jacobian to v_r , evaluated at the priori state estimate. The resulting expression is given by (9) [2]. The measurement matrix for the radar, $\mathbf{H}_r$, is given by (10). For the camera, the transformed (u, v) pixels directly provide us x , and y . Therefore, the $\mathbf{H}_c$ is given as expressed by (11). Note that r and c donate the radar and camera sensors, respectively.

$$\frac{\partial v_r}{\partial \mathbf{s}} = \left[\frac{y(v_y - xv_y)}{(x^2 + y^2)^{3/2}} \quad \frac{x(v_y - yv_x)}{(x^2 + y^2)^{3/2}} \quad \frac{x}{\sqrt{x^2 + y^2}} \quad \frac{y}{\sqrt{x^2 + y^2}} \right] \hat{\mathbf{s}}_{k-1} \quad (9)$$

$$\mathbf{H}_r = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \frac{y(v_y - xv_y)}{(x^2 + y^2)^{3/2}} & \frac{x(v_y - yv_x)}{(x^2 + y^2)^{3/2}} & \frac{x}{\sqrt{x^2 + y^2}} & \frac{y}{\sqrt{x^2 + y^2}} \end{bmatrix} \quad (10)$$

$$\mathbf{H}_c = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \quad (11)$$

3) Sensor Covariance Matrices

The autocovariance matrices define the measurement noise present and, accordingly, how confident the EKF should be in each sensor. The radar autocovariance is estimated using the approximation that the variances of phase, range, and Doppler are one-fourth of the radar's resolutions [2].

Since the radar's x and y measurement variances are required instead of range and azimuth, using the propagation of error given by (12) [10], the variance in the measurements in cartesian coordinates can be determined from the variances in the range and azimuth measurements in spherical coordinates. The variances of the x and y measurements are given in (13) and (14).

$$\sigma_z^2 = \left(\frac{\partial z}{\partial w} \right)^2 \cdot \sigma_w^2 + \left(\frac{\partial z}{\partial x} \right)^2 \cdot \sigma_x^2 + \dots \quad (12)$$

$$\sigma_x^2 = \cos^2(\theta) \cdot \left(\frac{\Delta \rho^2}{4} \right) + \rho^2 \sin^2(\theta) \cdot \left(\frac{\Delta \theta^2}{4} \right) \quad (13)$$

$$\sigma_y^2 = \sin^2(\theta) \cdot \left(\frac{\Delta \rho^2}{4} \right) + \rho^2 \cos^2(\theta) \cdot \left(\frac{\Delta \theta^2}{4} \right) \quad (14)$$

The radar autocovariance, $\mathbf{R}_r$ is therefore as given by (15). It is assumed that the noise present in each measurement are uncorrelated.

$$\mathbf{R}_r = \begin{bmatrix} \sigma_x^2 & 0 & 0 \\ 0 & \sigma_y^2 & 0 \\ 0 & 0 & \frac{\Delta v_r^2}{4} \end{bmatrix} \quad (15)$$

As for $\mathbf{R}_c$, camera measurement noise is assumed uncorrelated and dependent on the reprojection errors from the extrinsic calibration. Let Δd be the diagonal reprojection error; then, $\Delta u = \Delta v = \frac{\Delta d}{\sqrt{2}}$. The uncertainty in x and y can be computed as given in (16). The $\mathbf{R}_c$ expression is given by (17).

$$\begin{bmatrix} \Delta x \\ \Delta y \\ 1 \end{bmatrix} = \frac{1}{k} \times \mathbf{H}_{3 \times 3}^{-1} \times \begin{bmatrix} \Delta u \\ \Delta v \\ 1 \end{bmatrix} \quad (16)$$

$$\mathbf{R}_c = \begin{bmatrix} (\Delta x)^2 & 0 \\ 0 & (\Delta y)^2 \end{bmatrix} \quad (17)$$

4) Asynchronous Fusion and OOSM Handling

The selection between whether to apply synchronous or asynchronous fusion depends mainly on the specific application. In systems involving sensors with more reliable and similar update rates, it is common to apply synchronous fusion, which involves updating the Kalman filter using the measurements from all sensors simultaneously. While synchronous fusion is simpler, it may be less accurate and may introduce greater latency compared to asynchronous fusion [11].

Since (1) the YOLO object detection algorithm on the Raspberry Pi achieved an update rate of 70 FPS for a simulation video and, when limited by the used camera module frame rate, would achieve an update rate of 30 FPS, and (2) the radar module is configured with a 10 FPS frame rate, asynchronous fusion was opted for due to the relatively large update rate difference, avoiding inaccuracies and latency.

In asynchronous fusion, the $\mathbf{F}$ and $\mathbf{Q}$ matrices are time-variant, since they depend on Δt , calculated as $|t_{\text{current}} - t_{\text{meas_frame}}|$ [12], which is not constant for every iteration of the pipeline, making asynchronous fusion more computationally intensive. It also requires handling out-of-sequence measurements, which occur when the microprocessor receives a measurement from a sensor processing a frame at t_l , while the last EKF update corresponded to a frame taken at $t = k$, with $t_l < t_k$, meaning that the received measurement is from the past [12].

The simplest approach for handling OOSM is to simply discard stale measurements [13], but this risks losing valuable information. Therefore, we adopt the second-simplest method: which is to retrodict back to the time of the frame that the received measurement corresponds to [13]. The retrodicted a priori state is expressed by (18) [13]. The a priori estimation error covariance matrix remains referenced to time t_k , as given by (19), yet the process noise covariance matrix, $\mathbf{Q}_{k-1}$, is assumed to be zero [13].

$$\mathbf{s}_l^- = \mathbf{F}_{k-1}^{-1} \cdot \mathbf{s}_k^- \quad (18)$$

$$\mathbf{P}_k^- = \mathbf{F}_{k-1}^{-1} \mathbf{P}_{k-1}^+ (\mathbf{F}_{k-1}^{-1})^T \quad (19)$$

In the update step, the posteriori state is determined as given by (20). The Kalman gain and the posteriori estimation error autocovariance matrix remain unchanged and referenced to time t_k , as expressed by (21), and (22), respectively. While the method reduces the mathematical complexity of the equations, it falls short in accuracy, as it assumes when $\mathbf{Q}_{k-1}$ is zero, which is not an accurate assumption.

$$\mathbf{s}_k^+ = \mathbf{s}_k^- + \mathbf{K}_k (\mathbf{y}_l - \mathbf{H}_k \mathbf{s}_l^-) \quad (20)$$

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}_k^T \cdot (\mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^T + \mathbf{R}_k)^{-1} \quad (21)$$

$$\mathbf{P}_k^+ = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_k^- (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)^T + \mathbf{K}_k \mathbf{R}_k \mathbf{K}_k^T \quad (22)$$

D. Data Association and Track Management

The Kalman filter is a primary component in target tracking systems. Fig. 2 shows a block diagram of asynchronous target tracking framework. The flowchart is constructed based on [2], [13], [14]. Upon the arrival of a new measurement from a sensor, the measurement is processed (coordinate transformation and temporal alignment), and the prediction for sample step k is performed.

Assuming an (m) existing tracks, and (n) received measurements; for each track (m), the Jacobian of the function $h(\mathbf{s}_k)$, is used to map the track's EKF state predictions to the measurement prediction space. The process of gating, per track, is then performed to eliminate outlier measurements, thereby reducing the computations required in the association steps [13]. The data association method is then applied, followed by the track management logic, including initiation and deletion of tracks. After that, the EKF update step is performed for sample step k .

The data association method used was the (GNN) method. While it uses a deterministic approach, making it relatively less accurate compared to methods handling assignment probabilistically, it is considerably less computationally intensive on the Raspberry Pi and should provide acceptable performance in fairly complex scenarios.

1) Gating

As mentioned, gating is used to reduce the number of measurement-to-track associations considered [13], [15]. Gating is based on the Mahalanobis distance equation given by (23). Where $\mathbf{S}$ is the autocovariance matrix of the innovation term, $\mathbf{y}_j - \mathbf{H} \hat{\mathbf{s}}_i^-$. A measurement is said to be an outlier with 95% confidence based on the equation given by (24) [13], [15].

$$\mathbf{d}_{ij}^2 = (\mathbf{y}_j - \mathbf{H} \hat{\mathbf{s}}_i^-)^T \cdot \mathbf{S}^{-1} \cdot (\mathbf{y}_j - \mathbf{H} \hat{\mathbf{s}}_i^-) \quad (23)$$

$$(\mathbf{y}_j - \mathbf{H} \hat{\mathbf{s}}_i^-)^T \cdot \mathbf{S}^{-1} \cdot (\mathbf{y}_j - \mathbf{H} \hat{\mathbf{s}}_i^-) < G \quad (24)$$

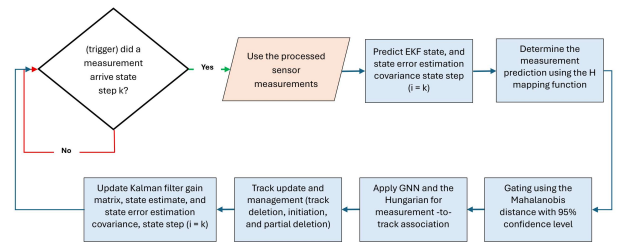


Fig. 2. High-level overview of the data fusion system.

The value of G is obtained from the chi-squared distribution with n degrees of freedom [13], [15]. In this implementation, three measurements are received from the radar, x , y , and v_r , and hence $n = 3$, where G is approximately 7.815. For the camera, $n = 2$, and G is approximately 5.991.

2) Global Nearest Neighbour (GNN)

After gating, the GNN finds the measurements-to-track association hypothesis (Ω^k) that has the highest probability, i.e., by maximizing the probability function, which can be shown to be by minimizing $\sum_{(i,j) \in \varphi_h} d_{ij}^2$ [13], [15], where φ_h is the data association hypothesis.

Practically, this corresponds to constructing a matrix filled by substituting the Mahalanobis distance when the measurement is within a track's gate, and 100 when the measurement is outside it [15], then determining the optimal assignment such that the corresponding distance sum is minimal. This is solved using the Hungarian algorithm. It finds the global optimal assignment in polynomial time $O(n^3)$ [16], rather than $O(\binom{n}{k})$.

3) Track Management

Track management involves a logic layer that handles partial track inactivation, deletion, and initiation [14]. For example, GNN may assign a track with an empty gate a measurement that is outside of its gate [15]. This is handled by flagging these tracks as partially inactive, and their rows are deleted from the constructed GNN matrix/cost function before the Hungarian algorithm is applied.

After the Hungarian step, measurements left unassigned are initiated as potential new tracks and tracks not assigned any measurement for 30 consecutive frames are deleted. When the assignment field exceeds 30, the track is said to be established and is displayed as a confirmed track. Fig. 3 provides a comprehensive description of the data association and track management logic for three cases: camera-alone, radar-alone, and combined EKF update steps. For the combined-update case, tracks are activated only when they are associated with both a radar and a camera measurement. This criterion reduces the risk of confirming tracks based on radar false alarm.

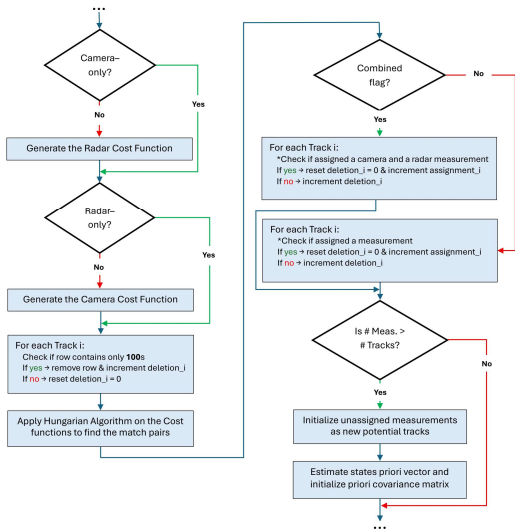


Fig. 3. Data association and track management flowchart.

E. Data Preparation and Track Structure Organization for Fusion Pipeline

Radar and camera measurements are sorted and organized before being input into the data fusion pipeline based on the timestamp at which the measurement frame was captured and the timestamp at which the Raspberry Pi received the measurement after processing (parsing for radar, YOLOv11 algorithm for camera). These timestamps are used to determine the EKF update flag and the OOSM flag.

F. Fog-Level Estimator ML Model

To attempt to improve the robustness of the system under foggy weather conditions, we aim to enable the model to estimate the fog-level and scale the camera sensor autocovariance matrix accordingly. Several methods based on [17] can be used for fog detection. The methods considered include:

- (1) Determining the dark channel prior
- (2) Calculating the contrast using the std of the image pixels
- (3) Determining the entropy, or randomness, present within the image pixels
- (4) Computing the global contrast factor
- (5) Measuring saturation, which conveys the colour level
- (6) Using Weber contrast

Influenced by the cited related work [3], we propose employing a simpler ML model that uses the six fog-detection features along with the average confidence score of YOLO-detected objects to classify the scene as having dense fog or no fog. The dataset found in [18] contains 1000 images categorized as High or No Fog.

Based on the classification, a mapping function, shown in Table II, is used to scale the camera autocovariance matrix, as given by (25). The updated camera autocovariance matrix is applied in the EKF update step, giving lower weight to the camera measurement under foggy conditions. The scaling factors can be tuned based on testing results. The Random Forest ML model was selected, as it is more comprehensible while still providing high accuracy [19].

$$\mathbf{R}_c = (1 + S.F.) \times \mathbf{R}_c \quad (25)$$

IV. EXPERIMENTAL SETUP

A. Hardware Model

The two sensors were mounted as close as possible to maximize the overlap of their fields of view. The radar antenna is positioned on its narrower side, which is the side kept closer to the camera.

TABLE II. RC SCALING FACTOR MAPPING FUNCTION

ML Model Classification Result	Scaling Factor (S.F.)
No Fog	0
High Fog	2

B. Calibration

The adopted calibration procedure is based on [20], and the overall intrinsic–extrinsic flow is shown in Fig. 4. The intrinsic parameters were determined using MATLAB’s Camera Calibrator toolbox. Checkerboard images were captured with the HQ-IR Cut camera and uploaded into the toolbox. The reprojection error was 0.21 px, as shown in Fig. 5(a). MATLAB’s tools also allow visualization of detected vs. the reprojected points, shown in Fig. 5(b).

To obtain the extrinsic parameters, we followed steps 4–7 of the flowchart presented in Fig. 4. A minimum of 20 images, along with their corresponding radar measurements of the radar reflector, must be collected. It is important to ensure a relatively significant change in the reflector position for each iteration. Fig. 6 illustrates examples of the gathered radar measurements, with the peak power point corresponding to the detected reflector.

It is necessary to confirm that the collected points are not planar, as this could result in degenerate solutions requiring additional points to be gathered. By visual inspection of the scatter plot shown in Fig. 7(a), we can verify that sufficient variation exists along all three axes. Following this, the next step involves obtaining the (u, v) pixel coordinates corresponding to the apex of the radar reflector in each image; an example is shown in Fig. 7(b).

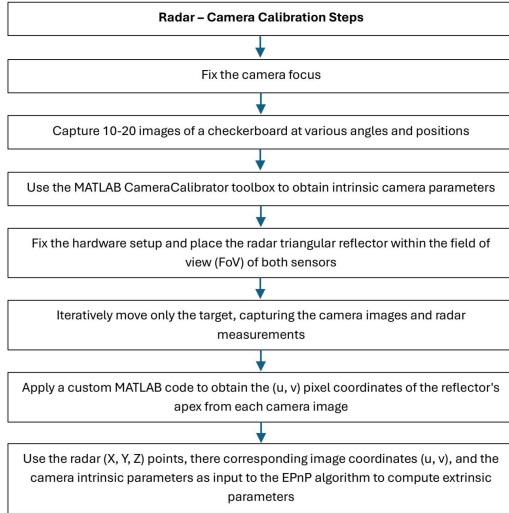


Fig. 4. Full Radar-Camera calibration steps.

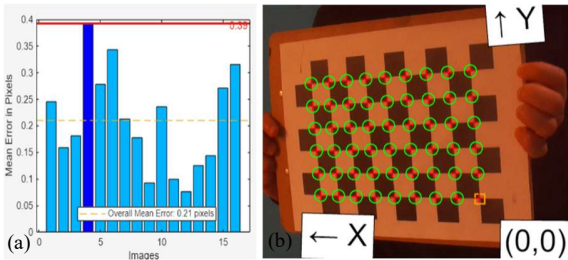


Fig. 5. Camera calibration results: (a) overall and reprojection error (in pixels) versus images chart alibration; (b) captured image with reprojected crosses plotted on the actual detected corners

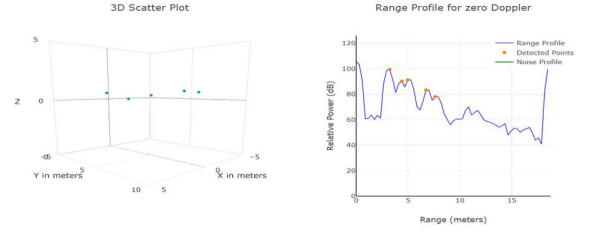


Fig. 6. Radar measurement for a reflector in an extrinsic calibration setup on mmWave_Demo_Visualizer webpage

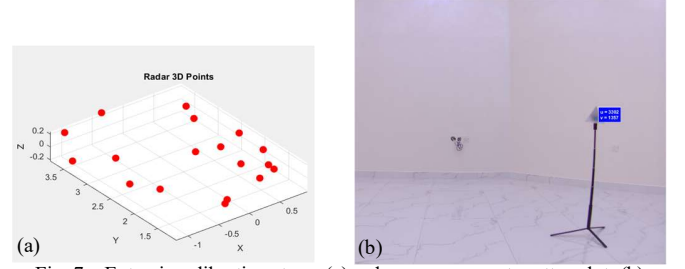


Fig. 7. Extrinsic calibration steps: (a) radar measurement scatter plot; (b) Image showing the pixel point corresponding to the apex of the corner reflector.

With the radar points, their corresponding pixel coordinates, and the intrinsic camera parameters available, the Efficient Perspective-n-Point (EPnP) algorithm can be executed to obtain the rigid body transformation matrix. A link to the GitHub repository containing the (EPnP) code is provided in [21].

C. Selected Unified Fusion PoV

Having obtained the rigid body transformation matrix, we first select the unified fusion PoV before proceeding further. In unified FV, the 3D radar measurements are projected onto the camera image plane using the projective transformation matrix. In unified BEV, however, the 2D image pixel coordinates are projected onto the radar 2D coordinate plane using inverse projective mapping, or the inverse of the homography matrix [1], [2]. While FV may seem superior to BEV because it preserves depth information, BEV is preferred in tracking applications, as it is more robust against the crucial challenge of occlusion [1].

D. Applying Transformation Matrices: Ground Truth Data and YOLO Centroids

The extracted ArUco marker 3D coordinates are from the camera coordinate frame perspective. To obtain 3D BEV coordinates respective points of the ground truth, we apply the inverse rigid-body transformation from the camera frame to the radar frame. Since the rotational matrix is orthogonal, its inverse equals its transpose. The transformation is expressed in (26) [6], where $R_{c \rightarrow r}$ and $t_{c \rightarrow r}$ are as given in (27) and (28).

$$\begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = R_{c \rightarrow r} \begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix} + t_{c \rightarrow r} \quad (26)$$

$$R_{c \rightarrow r} = R_{r \rightarrow c}^{-1} = R_{r \rightarrow c}^T \quad (27)$$

$$t_{c \rightarrow r} = -R_{r \rightarrow c}^{-1} \times t_{r \rightarrow c} \quad (28)$$

To obtain the radar measurements from the YOLO centroid measurements, the pixel coordinates were first resized back to the original resolution, undoing the resizing applied during YOLO implementation. Next, the inverse of the homography transformation matrix was applied, and the corresponding (x, y) coordinates were computed in the same way as shown in (17). The coordinates are normalized by the constant k , corresponding to the third term of the radar homogeneous coordinates.

E. Summary of Key Engineering Decisions and Rationale

Table III summarizes some of the key engineering decisions made, the options considered, and the rationale for the final selection.

V. RESULTS AND DISCUSSION

A. Calibration

The extrinsic calibration EPnP algorithm estimated the rigid body transformation matrix with an achieved reprojection error of 54.365 px. Although this may appear high, the image resolution is 4056×3040 , and the fusion operates at the object level. Normalizing relative to the diagonal of the image gives 0.0107, approximately 1% error, which is acceptable.

B. Applying Transformation Matrices

After the application of the calibration matrices on the ground-truth dataset, the resulting coordinates were reasonable and closely matched the radar measurements, validating the extrinsic and intrinsic calibration procedures and matrices. As for the YOLO centroid dataset, when choosing $Z_{projection}$ as zero, meaning all the obtained YOLO detection centroids are projected into the $Z = 0$ plane in the radar coordinate frame, the respective radar points were not reasonable, with negative y -values. Adjusting Z further worsened the errors.

TABLE III. SUMMARY OF SENGINEERING DECISIONS AND THERE RATIONALE

Design Decision	Selected Option	Alternatives Considered	Selection Rationale
Unified coordinate frame	Radar BEV	Camera FV	BEV is better for tracking and occlusion handling
Ground-truth acquisition method	ArUco Markers	LiDAR, RTK-GPS	Selected for its low cost, indoor deployability, and sufficient accuracy
Fusion framework	Asynchronous EKF	Synchronous EKF	Since Radar: 10 FPS, YOLO: 30 FPS, it avoids latency and inaccuracy
Motion model	CA model, winer-sequence variant	CV	Strikes a balance between accuracy and complexity.
Data association	GNN+ Hungarian	PDA, JPDA	Selected for its lower complexity, computational load, and latency
Fog classifier ML model	Random Forest	ANN	Interpretable, lighter, and requires smaller training datasets

Since the ground-truth results were valid, the issue was determined to be the homography approximation rather than the calibration process. Deviations from the assumed plane increase error. In our implementation, the model was placed on a table. In future implementations, the model should be placed on the ground, as observed in [2], so that the scene better conforms to the homography approximation.

C. Fog-Level Estimator ML Model

From the dataset cited in [20], the seven visual features listed previously were extracted. Features that had almost no variance or were highly correlated were removed to reduce redundancy. The remaining features were scaled to a 0–1 range using MinMax normalization. The learning curve, shown in Fig. 8, showed that the training and validation accuracies were close, indicating that the model was not overfitting. The final test set confirmed generalization; the performance metric scores are given in Table IV.

D. Model Tracking Pipeline Results

To validate the system, multiple scenes capturing multithreaded camera and radar data were recorded. The fusion plotting module outputs animations of ArUco marker measurements, tracker output, YOLO output, RMSE per frame, and radar measurements, including x vs. y and range vs. radial velocity. Fine-tuning was required for the deletion increment threshold, initialized covariance matrix, initialized tracker states, standard deviation of the target acceleration in x and y , and radar measurement gating threshold. This fine-tuning was completed for one of the scenes, with the resulting animation given in the linked video: <https://youtu.be/H9NKajhr28o>. The RMSE plot for the tested scene is shown in Fig. 9.

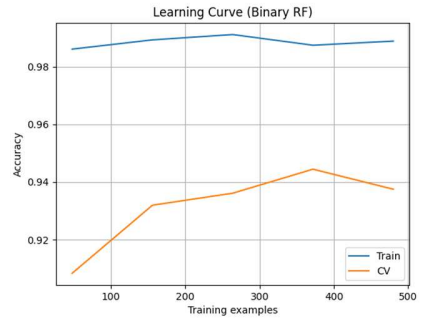


Fig. 8. Learning curve of the Random Forest, showing training and validation accuracy versus the epochs.

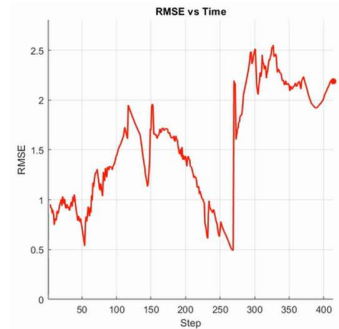


Fig. 9. Outout RMSE plot of the tested scene

TABLE IV. PERFORMANCE RESULTS OF THE TESTING DATASET

	Testing Dataset		
	Precision	Recall	F1-score
Dense	0.98	0.96	0.97
No Fog	0.96	0.98	0.97
Accuracy	0.97		

E. Limitations and Failure Analysis

It should be noted that the tracking validation is limited by the assumed accuracy of the ArUco-based ground-truth measurements. While ArUco markers provide a practical reference, their localization accuracy may degrade as the target distance from the camera increases, which can affect the computed RMSE values.

RMSE spikes correspond to frames where ArUco markers were not visible. Some frames also showed the radar detecting only static objects, even when a moving target was present. In other frames, RMSE reached values as low as 0.5 m.

As for the data association performance, when a second target enters from the same side as the first target exits, the GNN tracker assumes it is the same target and continues along its path for some frames. Another track is established only after a few frames. Consequently, the RMSE toward the end reflects errors between mismatched ground-truth targets and track IDs, as the tracker continued comparing the first established track states with the plotted ground-truth values of the second target, explaining the higher RMSE spikes toward the end.

Moreover, after correcting the homography transformation, the fused transformed YOLO detections should support the tracker in producing more accurate measurements. The GNN data association is then expected to be more reliable, distinguishing closer tracks and resulting in more stable tracker output and lower RMSE.

VI. CONCLUSION

In this paper, the design and engineering considerations for a radar-camera fusion framework were presented. The implemented framework validates the radar-only tracking, calibration pipeline, and fog-classification module. Full quantitative evaluation of radar-camera fusion and potential improved system performance under foggy conditions remains limited by the experimental homography setup, highlighting practical calibration challenges that should be addressed in future testing. Moreover, the system implementation is available at <https://surl.lu/fqwxft>.

REFERENCES

- [1] S. Yao et al., "Radar-Camera Fusion for Object Detection and Semantic Segmentation in Autonomous Driving: A Comprehensive Review," arXiv (Cornell University), Apr. 2023, doi: <https://doi.org/10.1109/tiv.2023.3307157>.
- [2] R. Zhang and S. Cao, "Extending Reliability of mmWave Radar Tracking and Detection via Fusion With Camera," IEEE Access, vol. 7, pp. 137065–137079, 2019, doi: <https://doi.org/10.1109/access.2019.2942382>.
- [3] Z. Li, S. Zhang, Z. Fu, F. Meng, and L. Zhang, "Confidence-Feature Fusion: A Novel Method for Fog Density Estimation in Object Detection Systems," Electronics, vol. 14, no. 2, pp. 219–219, Jan. 2025, doi: <https://doi.org/10.3390/electronics14020219>.
- [4] "mmWave Demo Visualizer User's Guide mmWave Demo Visualizer," 2017. Accessed: Mar. 20, 2025. [Online]. Available: https://www.ti.com/lit/ug/swru529c/swru529c.pdf?ts=1742455933180&ref_url=https%253A%252F%252Fwww.google.com%252F
- [5] M. F. Mohamad Ali, M. S. Abustan, S. H. Abu Talib, I. Abustan, N. Abd Rahman, and H. Gotoh, "A Case Study on the Walking Speed of Pedestrian at the Bus Terminal Area," E3S Web of Conferences, vol. 34, p. 01023, 2018, doi: <https://doi.org/10.1051/e3sconf/20183401023>.
- [6] Mahdi Chamseddine, J. Rambach, and D. Stricker, "CaRaCTO: Robust Camera-Radar Extrinsic Calibration with Triple Constraint Optimization," pp. 534–545, Jan. 2024, doi: <https://doi.org/10.5220/0012369700003654>.
- [7] R. Hartley, A. Zisserman, S. Wei, and Quanbing Zhang, 计算机视觉中的多视图几何 = Multiple view geometry in computer vision / Ji suan ji shi jue zhong de duo shi tu ji he = Multiple view geometry in computer vision. 机械工业出版社, Beijing: Ji Xie Gong Ye Chu Ban She, 2020.
- [8] B. Fisher, "Projective Transformations," homepages.inf.ed.ac.uk, Nov. 07, 1997. https://homepages.inf.ed.ac.uk/rbf/CVonline/LOCAL_COPIES/BEARD_SLEY/node3.html
- [9] X. Li and V. P. Jilkov, "Survey of maneuvering target tracking: dynamic models," Jul. 2000, doi: <https://doi.org/10.1117/12.391979>.
- [10] V. Lindberg, "Uncertainties and Error Propagation," www.geol.lsu.edu, Jul. 01, 2000. <http://www.geol.lsu.edu/jlorenzo/geophysics/uncertainties/Uncertaintiespart2.html>
- [11] K. Zhang, Z. Wang, L. Guo, Y. Peng, and Z. Zheng, "An Asynchronous Data Fusion Algorithm for Target Detection Based on Multi-Sensor Networks," IEEE Access, vol. 8, pp. 59511–59523, 2020, doi: <https://doi.org/10.1109/access.2020.2982682>.
- [12] Kalman, "Signal Processing Stack Exchange," Signal Processing Stack Exchange, Sep. 04, 2019. <https://dsp.stackexchange.com/questions/60511/kalman-filter-how-to-combine-data-from-sensors-with-different-measurement-rate/60513#60513> (accessed Aug. 26, 2025).
- [13] D. Waard, "UvA-DARE (Digital Academic Repository) A new approach to distributed data fusion," 2008. Accessed: Aug. 26, 2025. [Online]. Available: https://pure.uva.nl/ws/files/4318786/59162_07.pdf
- [14] K. Aziz, Eddy De Greef, Maxim Rykunov, A. Bourdoux, and H. Sahli, "Radar-camera Fusion for Road Target Classification," Sep. 2020, doi: <https://doi.org/10.1109/radarconf2043947.2020.9266510>.
- [15] Pavlina Konstantinova, A. Udvarov, and Tzvetan Semerdjiev, "A study of a target tracking algorithm using global nearest neighbor approach," Jan. 2003, doi: <https://doi.org/10.1145/973620.973668>.
- [16] Wikipedia Contributors, "Hungarian algorithm," Wikipedia, Aug. 27, 2019. https://en.wikipedia.org/wiki/Hungarian_algorithm
- [17] R. Castelli, P. Frolkovic, C. Reinhardt, C. C. Stolk, J. Tomczyk, and A. Vromans, "Fog detection from camera images," pp. 25–43, Jan. 2016.
- [18] Yessica, "Foggy Cityscapes Images," Kaggle.com, 2025. <https://www.kaggle.com/datasets/yessicatuteja/foggy-cityscapes-image-dataset> (accessed Nov. 23, 2025).
- [19] P. Roßbach, "Neural Networks vs. Random Forests – Does it always have to be Deep Learning?," frankfurt. <https://blog.frankfurt-school.de/wp-content/uploads/2018/10/Neural-Networks-vs-Random-Forests.pdf> (accessed Nov. 23, 2025).
- [20] D. Kim and S. Kim, "Extrinsic parameter calibration of 2D radar-camera using point matching and generative optimization," pp. 99–103, Oct. 2019, doi: <https://doi.org/10.23919/icc47443.2019.8971568>.
- [21] Adynathos, "EPnP: An Accurate O(n) Solution to the PnP Problem," Github.com, 2025. <https://github.com/cvlab-epfl/EPnP>